
Hardware-Aware Optimization and Benchmarking of Generative Transformers on Edge ARM Architectures

Daniel Sanei

Department of Electrical and Computer Engineering
University of California, San Diego
dsanei@ucsd.edu

Abstract

This project investigates the performance trade-offs of deploying autoregressive deep generative models on resource-constrained edge hardware. Using the Raspberry Pi 5 as a target platform, we evaluate how quantization level and inference configuration parameters impact inference latency, sustained throughput, and generative fidelity of transformer-based architectures. We provide a detailed analysis of the relationship between memory bandwidth, PMIC power draw, thermal behavior, and the structural integrity of the learned probability distribution under compression.

1 Research Problem

The primary research problem is the architectural and computational bottleneck of deploying high-parameter Deep Generative Models (DGMs) on the edge. Autoregressive transformer models are fundamentally bound by the memory bandwidth required to map complex joint probability distributions $P(x_1, \dots, x_T)$ in real-time. On low-power devices like the Raspberry Pi 5, the lack of dedicated tensor cores necessitates a transition from traditional GPU-bound paradigms to CPU-centric optimizations.

The “Memory Wall” is particularly acute for DGMs because each token generation step requires reading the entire model weight set from RAM. This bottleneck is exacerbated by the sequential nature of autoregressive decoding, where computational intensity is low but data movement is massive. For a 7B parameter model in a compressed 4-bit state, the system must move approximately 3.5–4GB of weight data through the memory bus per token generated. This project investigates the performance floor of these generative distributions by implementing post-training quantization (PTQ) and evaluating how mapping models to low-precision integer spaces affects both generative fidelity and hardware-level stability.

2 Importance

As generative AI moves toward local deployment in robotics and embedded systems, understanding the implementation of these models on low-power silicon is critical. The challenge lies in maintaining a model’s generative capabilities while adhering to strict thermal and power envelopes. Since typical desktop GPUs are unsuitable for portable applications, we must optimize DGM architectures to run locally. This work helps define hardware-aware requirements for edge-deployed generative applications by analyzing the efficiency of lightweight ARM processors in sustaining the workloads required for transformer inference, without relying on cloud-based infrastructure or incurring the associated latency and privacy trade-offs.

3 Existing Works and Limitations

Existing research typically focuses on deploying transformers on high-end desktop GPUs or specialized Apple Silicon. While there are works deploying lightweight models like Phi-3 or Gemma-2B on Single-Board Computers (SBCs) using quantization (INT8/INT4), they often prioritize raw accuracy over hardware-level efficiency on ARM architectures. Recent evaluations by Nezami et al. (1) on Raspberry Pi 5 clusters identified bottlenecks in the decode phase that limit scalability for models with large context windows. Similarly, Yoon et al. (2) demonstrated INT8 quantization on the Pi 4 to improve speed, but the study lacked ARM-specific cache analysis and power profiling. There is a clear need for a systematic study that correlates model compression and runtime configuration with real-time thermal and power metrics on modern edge hardware.

4 Inference Framework and Software Stack

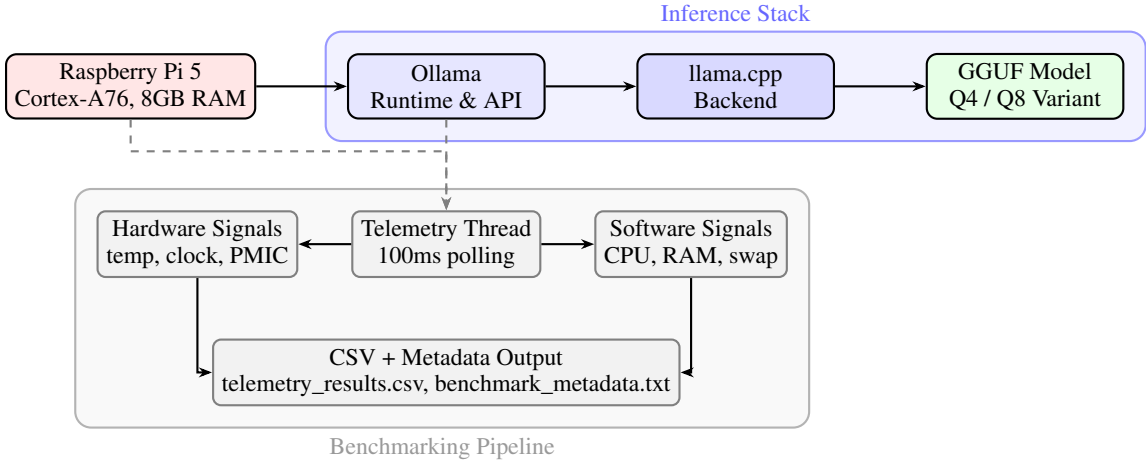


Figure 1: System architecture of the benchmarking environment. The inference stack (Ollama → llama.cpp → GGUF) runs on the Pi 5 while a parallel telemetry thread samples hardware and software signals every 100ms.

4.1 The Ollama and llama.cpp Backend

We utilize the Ollama framework, an abstraction layer built on the llama.cpp backend. Ollama manages model weights using the GGUF format, which is designed for unified memory architectures where the CPU and system RAM share a common pool, matching the Pi 5’s memory layout. The framework exposes an API returning nanosecond-level timing metadata, enabling precise measurement of the prefill and decoding phases. All optimization parameters in this study are passed as runtime configuration options, requiring no modifications to model weights or inference code.

4.2 Model Selection

Model selection was primarily constrained by the Pi 5’s 8GB RAM ceiling. At Q4 quantization, a 13B model would require approximately 8GB on its own, leaving no headroom for the KV cache or system processes. A 7B model at Q8 quantization similarly approaches the limit. This effectively bounds the practical deployment range to models under 8B parameters at Q4, or under 4B at Q8, ruling out larger architectures entirely without further compression. Within these constraints, we selected three models that span the feasible range:

1. **Llama 3.2 (1B):** A lightweight baseline for maximum achievable throughput, where the small memory footprint isolates CPU performance as the primary variable.
2. **Phi-4 Mini (3.8B):** A higher-reasoning model that tests the Pi’s ability to handle a more complex architecture while remaining within the RAM envelope at both quantization tiers.

3. **Llama 3.1 (8B)**: A stress-test model at the upper limit of sustainable inference on this platform. At Q4 quantization, its footprint is approximately 5–6GB, leaving limited headroom for the KV cache, and at Q8 it nearly saturates the 8GB ceiling entirely.

5 Methods

5.1 Mathematical Setup

The models under evaluation are autoregressive decoder-only transformers, which factorize the joint probability distribution of a sequence as follows:

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_1, \dots, x_{t-1}; \theta) \quad (1)$$

Token generation involves iterative sampling from the Softmax output, controlled by a temperature parameter τ :

$$P(x_t = i \mid x_{<t}) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \quad (2)$$

Our benchmarks evaluate the hardware’s ability to compute these distributions across billions of parameters in real-time, focusing on sustained throughput during the autoregressive decoding phase.

5.2 Quantization and KV Cache

Following Dettmers et al. (3), we apply Post-Training Quantization (PTQ) to reduce 16-bit floating-point weights to lower-precision integer representations. We compare two quantization tiers: Q8_0 (8-bit) and Q4_K_M (4-bit), selected as GGUF model variants within Ollama. The linear quantization mapping with scaling factor s and zero-point z is:

$$w_q = \text{round} \left(\frac{1}{s} \cdot w_f + z \right) \quad (3)$$

To optimize decoding, `llama.cpp` employs a KV Cache that stores self-attention keys and values across generation steps rather than recomputing them, reducing the per-step attention computation from $O(n^2)$ to $O(n)$ per newly generated token. However, the cache grows linearly with sequence length, introducing memory pressure that compounds with model size. On the Pi 5, we monitor RAM growth to evaluate whether the 8GB ceiling can sustain longer generations without triggering swap usage.

5.3 Inference Configuration Parameters

In addition to quantization, `llama.cpp` exposes runtime parameters that can affect performance on memory-constrained hardware, configured via the Ollama API options dictionary:

```
response = client.generate(model=model, prompt=prompt,
    options={"num_ctx": 2048, "num_thread": 4})
```

Context Window Size (`num_ctx`) controls the maximum token context and directly determines KV cache size. We evaluate sizes of 512, 2048, and 4096 tokens. Smaller windows reduce RAM pressure and can improve TPS, but limit the model’s reasoning over longer inputs.

Thread Count (`num_thread`) controls how many of the Pi 5’s four Cortex-A76 cores are used for inference. We compare 2-thread and 4-thread configurations. Since autoregressive decoding is memory-bandwidth-bound rather than compute-bound, we expect diminishing returns beyond a small thread count, which our telemetry will help confirm.

5.4 Generative Fidelity and Perplexity

To quantify the quality cost of compression, we use Perplexity (PPL), the exponentiated average negative log-likelihood of a sequence:

$$\text{PPL}(X) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(x_i \mid x_{<i}) \right) \quad (4)$$

Comparing PPL between Q8 and Q4 variants on identical reference texts quantifies the fidelity cost of the compression gain.

PPL computation requires access to per-token log-probabilities, which the Ollama API does not expose directly. We therefore use `llama-cpp-python` (4), a well-maintained Python binding for `llama.cpp` that exposes the same GGUF model weights and quantization tiers used by Ollama, while providing direct access to token-level log-probabilities via the `logprobs` parameter. This allows PPL to be computed consistently across all model and quantization combinations without modifying the underlying model or weights:

```
from llama_cpp import Llama

llm = Llama(model_path="model.gguf", logits_all=True)

def compute_ppl(llm, text):
    output = llm(text, max_tokens=0, logprobs=True, echo=True)
    log_probs = [t['logprob'] for t in
                  output['choices'][0]['logprobs']['token_logprobs']]
    if t is not None]
    return math.exp(-sum(log_probs) / len(log_probs))
```

PPL is computed on prompts 1–4, which produce natural language outputs where token-level probability is a meaningful quality signal. Prompt 5 (mathematical reasoning) produces a verifiable, deterministic output that can be checked directly for correctness, making PPL less informative as a standalone metric for that task.

5.5 Performance Metrics and Telemetry

The following metrics are recorded for each model and configuration combination: Time to First Token (TTFT) representing prefill latency, Tokens Per Second (TPS) for sustained decoding throughput, and peak RAM consumption tracking both model weight residency and KV cache growth. A multithreaded Python script samples hardware state every 100ms in parallel with each inference call, recording the following signals:

- **Power:** Total board wattage estimated via `vcgencmd pmic_read_adc`, summing $P = \sum I_i V_i$ across all PMIC rails.
- **Thermals:** Die temperature via `vcgencmd measure_temp` and throttle status via `vcgencmd get_throttled`. Throttling begins at approximately 80°C on the Cortex-A76.
- **Clock Speed:** ARM core frequency via `vcgencmd measure_clock arm` to detect frequency scaling events.
- **CPU and RAM:** System-wide load and memory usage via `psutil`.

Timing metrics (TTFT, TPS, total latency) are extracted from the Ollama API response, which provides nanosecond-resolution timing for the prefill and decode phases separately.

6 Experimental Design

All tests use a fixed random seed and temperature of 0.0 for deterministic, reproducible outputs. The prompt bank below covers a range of generative task types. Perplexity is computed on prompts 1–4; for prompt 5, the model’s derivation and numerical result are verified directly for correctness.

1. **Technical/Conceptual:** “Is Python an interpreted or compiled language?”
2. **Creative Writing:** “Write a short story about a robot discovering a garden on a deserted planet.”
3. **Summarization:** “Summarize the key differences between ARM and x86 architectures in two paragraphs.”
4. **Logical Reasoning:** “A farmer has 17 sheep. All but 9 run away. How many sheep does the farmer have left? Explain your reasoning step by step.”

5. **Mathematical Reasoning:** “Derive the formula for the sum of an arithmetic series and verify it for the integers 1 through 10.”

7 Preliminary Results

7.1 Initial Benchmark: Llama 3.2 (1B), Q4 Quantization

As an initial validation of the benchmarking pipeline, we ran Prompt 1 against Llama 3.2 1B at Q4 quantization. Inference metrics from the Ollama API are shown in Table 1.

Metric	Value
Total Latency	54.75s
Model Load Duration	2.77s
Time to First Token (TTFT)	1.21s
Prompt Tokens	33
Prompt Eval Rate	27.24 tokens/s
Generated Tokens	256
Token Generation Rate (TPS)	5.07 tokens/s

Table 1: Inference performance for Llama 3.2 1B (Q4) on Prompt 1.

The telemetry data reveals a clear two-phase hardware behavior. During model load, power draw was approximately 3.5W with moderate CPU load (around 30%). Once prefill began, CPU load saturated to 100% across all cores and power jumped to a sustained 6–8W, consistent with the memory-bandwidth-bound nature of CPU-only inference. Temperature rose steadily from 45°C to a peak of 58–59°C, well below the 80°C throttling threshold. No throttling events were recorded and no swap was used, confirming the 1B model fits comfortably within the 8GB RAM envelope. RAM stabilized at approximately 2.7GB during decoding. The model response was qualitatively correct, confirming the pipeline is functioning as expected.

7.2 Next Steps

With the benchmarking pipeline validated, the remaining work proceeds as follows:

1. **Perplexity Implementation:** Integrate PPL scoring for prompts 1–5 using `llama-cpp-python` to extract per-token log-probabilities directly from the GGUF model weights, consistent across all model and quantization pairs.
2. **Full Sweep:** Run the complete prompt bank across all three models at both Q4 and Q8 quantization tiers, with context window sizes of 512, 2048, and 4096, and thread counts of 2 and 4, collecting the full set of performance and telemetry metrics.
3. **Comparative Analysis:** Analyze trade-offs across all configuration axes, with particular attention to how context window size strains the memory envelope for the larger models, and whether thread count yields meaningful throughput gains given the memory-bound nature of the decode phase.

8 Discussion

8.1 Novelty and Advantages

The primary contribution of this work is its hardware-software co-design focus on a real commodity edge platform. Most DGM benchmarking targets high-end GPU hardware; this study profiles how model compression and runtime configuration interact with ARM-specific thermal limits and PMIC power rails. By correlating inference phases with hardware telemetry at 100ms resolution, we can identify where energy is spent during prefill versus decoding, which is directly relevant to battery-operated robotics applications.

8.2 Limitations

The inability to use GPU acceleration on the Pi 5 means our results represent the CPU-only performance floor for this class of hardware. The 8GB RAM ceiling also prevents evaluation of full-precision FP16 models for the larger architectures, limiting the quantization comparison to Q4 versus Q8 tiers. Finally, perplexity computed from the model’s own outputs is a self-referential quality metric; future work could compare against an external reference model for a more absolute fidelity measure.

References

- [1] Nezami, Z., et al. “Generative AI on the Edge: Architecture and Performance Evaluation.” arXiv:2411.17712, 2024.
- [2] Yoon, K., et al. “LiteRT-Optimized INT8 LLM for Raspberry Pi4 Deployment.” ICCV Workshops, 2025.
- [3] Dettmers, T., et al. “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.” NeurIPS, 2022.
- [4] Andrei, A. “llama-cpp-python: Python Bindings for llama.cpp.” <https://github.com/abetlen/llama-cpp-python>, 2023.